

Assignment 2 — k-NN on UniversalBank

Jaipreet Kaur

September 28, 2025

Note: This version uses `check.names = FALSE` when reading the CSV so that column names like `ZIP Code` and `Personal Loan` remain unchanged. This avoids errors when subsetting columns.

Setup

```
# install.packages(c("tidyverse", "class", "knitr", "rmarkdown", "tinytex")) # if needed
library(dplyr)
library(ggplot2)
library(class)
set.seed(42)
```

Load & Preprocess

```
# Keep original column names with spaces
bank <- read.csv("UniversalBank.csv", stringsAsFactors = FALSE, check.names = FALSE)

dat <- bank %>%
  select(-ID, -`ZIP Code`)

# One-hot encode Education into 3 dummies
edu_mm <- model.matrix(~ factor(Education) - 1, data = dat) %>% as.data.frame()
colnames(edu_mm) <- c("Education_1", "Education_2", "Education_3")

dat2 <- dat %>%
  select(-Education) %>%
  bind_cols(edu_mm)

y <- dat2$`Personal Loan` # target
X <- dat2 %>% select(-`Personal Loan`)

str(X)
```

```
## 'data.frame':    5000 obs. of  13 variables:
##  $ Age           : int  25 45 39 35 35 37 53 50 35 34 ...
##  $ Experience     : int   1 19 15  9  8 13 27 24 10  9 ...
##  $ Income         : int  49 34 11 100 45 29 72 22 81 180 ...
```

```
## $ Family      : int  4 3 1 1 4 4 2 1 3 1 ...
## $ CCAvg       : num  1.6 1.5 1 2.7 1 0.4 1.5 0.3 0.6 8.9 ...
## $ Mortgage    : int  0 0 0 0 0 155 0 0 104 0 ...
## $ Securities Account: int  1 1 0 0 0 0 0 0 0 0 ...
## $ CD Account  : int  0 0 0 0 0 0 0 0 0 0 ...
## $ Online      : int  0 0 0 0 0 1 1 0 1 0 ...
## $ CreditCard  : int  0 0 0 0 1 0 0 1 0 0 ...
## $ Education_1 : num  1 1 1 0 0 0 0 0 0 0 ...
## $ Education_2 : num  0 0 0 1 1 1 1 0 1 0 ...
## $ Education_3 : num  0 0 0 0 0 0 0 1 0 1 ...
```

```
table(y)
```

```
## y
##   0    1
## 4520  480
```

Helper Functions

```
scale_train <- function(X_train) {
  mu <- sapply(X_train, mean)
  sdv <- sapply(X_train, sd)
  list(mu=mu, sd=sdv)
}
scale_apply <- function(X_df, mu, sdv) {
  as.data.frame(scale(as.matrix(X_df), center = mu, scale = sdv))
}

conf_mat_acc <- function(y_true, y_pred) {
  tab <- table(Actual=y_true, Pred=y_pred)
  acc <- mean(y_true == y_pred)
  list(cm=tab, acc=acc)
}
```

60/40 Split, k Sweep

```
set.seed(42)
idx <- sample(seq_len(nrow(X)), size = floor(0.60*nrow(X)))
X_train <- X[idx, , drop=FALSE]
y_train <- y[idx]
X_val   <- X[-idx, , drop=FALSE]
y_val   <- y[-idx]

stats <- scale_train(X_train)
X_train_s <- scale_apply(X_train, stats$mu, stats$sd)
X_val_s   <- scale_apply(X_val,   stats$mu, stats$sd)
```

```

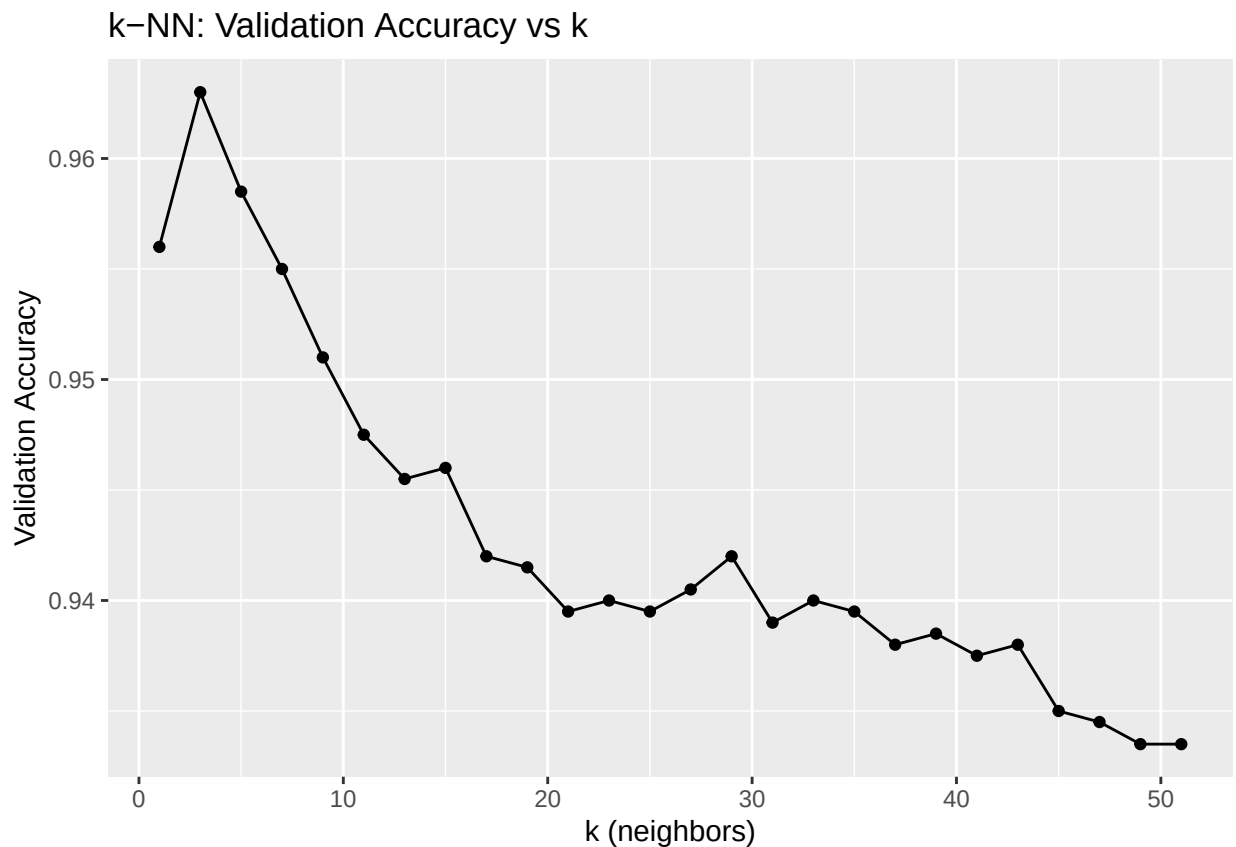
ks <- seq(1, 51, by=2)
accs <- numeric(length(ks))

for (i in seq_along(ks)) {
  k <- ks[i]
  pred <- knn(train = X_train_s, test = X_val_s, cl = factor(y_train, levels=c(0,1)), k = k, prob = TRUE)
  accs[i] <- mean(as.integer(as.character(pred)) == y_val)
}

best_k <- ks[which.max(accs)]
best_acc <- max(accs)

data.frame(k=ks, accuracy=accs) %>%
  ggplot(aes(k, accuracy)) + geom_line() + geom_point() +
  labs(title="k-NN: Validation Accuracy vs k", x="k (neighbors)", y="Validation Accuracy")

```



Best k: 3 with validation accuracy 0.963.

```

pred_best <- knn(X_train_s, X_val_s, cl=factor(y_train, levels=c(0,1)), k=best_k, prob=TRUE)
res <- conf_mat_acc(y_val, as.integer(as.character(pred_best)))
res$cm; res$acc

```

```

##      Pred
## Actual  0   1
##      0 1799   9
##      1   65 127

```

```
## [1] 0.963
```

Customer Classification

```
cust <- data.frame(
  Age=40, Experience=10, Income=84, Family=2, CCAvg=2,
  Mortgage=0, `Securities Account`=0, `CD Account`=0, Online=1, CreditCard=1,
  Education_1=0, Education_2=1, Education_3=0, check.names=FALSE
)[, colnames(X)]

cust_s <- scale_apply(cust, stats$mu, stats$sd)

# k=1
pred_k1 <- knn(X_train_s, cust_s, cl=factor(y_train, levels=c(0,1)), k=1, prob=TRUE)
p_yes_k1 <- attr(pred_k1, "prob")
pred_class_k1 <- as.integer(as.character(pred_k1))

# k=best_k
pred_kb <- knn(X_train_s, cust_s, cl=factor(y_train, levels=c(0,1)), k=best_k, prob=TRUE)
p_yes_kb <- attr(pred_kb, "prob")
pred_class_kb <- as.integer(as.character(pred_kb))

data.frame(
  setting = c("k=1", "best k"),
  predicted_class = c(pred_class_k1, pred_class_kb),
  prob_for_winning_class = c(p_yes_k1, p_yes_kb)
)

##      setting predicted_class prob_for_winning_class
## 1      k=1              0              1
## 2 best k              0              1
```

50/30/20 Split

```
set.seed(42)
n <- nrow(X)
test_idx <- sample(seq_len(n), size = floor(0.20*n))
X_test <- X[test_idx, , drop=FALSE]; y_test <- y[test_idx]
X_rem <- X[-test_idx, , drop=FALSE]; y_rem <- y[-test_idx]

train2_idx_rel <- sample(seq_len(nrow(X_rem)), size = floor(0.625*nrow(X_rem)))
X_train2 <- X_rem[train2_idx_rel, , drop=FALSE]; y_train2 <- y_rem[train2_idx_rel]
X_val2 <- X_rem[-train2_idx_rel, , drop=FALSE]; y_val2 <- y_rem[-train2_idx_rel]

stats2 <- scale_train(X_train2)
X_train2_s <- scale_apply(X_train2, stats2$mu, stats2$sd)
X_val2_s <- scale_apply(X_val2, stats2$mu, stats2$sd)
X_test_s <- scale_apply(X_test, stats2$mu, stats2$sd)
```

```

pred_tr <- knn(X_train2_s, X_train2_s, cl=factor(y_train2, levels=c(0,1)), k=best_k)
pred_val <- knn(X_train2_s, X_val2_s, cl=factor(y_train2, levels=c(0,1)), k=best_k)
pred_te <- knn(X_train2_s, X_test_s, cl=factor(y_train2, levels=c(0,1)), k=best_k)

res_tr <- conf_mat_acc(y_train2, as.integer(as.character(pred_tr)))
res_val <- conf_mat_acc(y_val2, as.integer(as.character(pred_val)))
res_te <- conf_mat_acc(y_test, as.integer(as.character(pred_te)))

list(
  Train_Accuracy = res_tr$acc,
  Validation_Accuracy = res_val$acc,
  Test_Accuracy = res_te$acc
)

```

```

## $Train_Accuracy
## [1] 0.9768
##
## $Validation_Accuracy
## [1] 0.9613333
##
## $Test_Accuracy
## [1] 0.964

```

```
cat("Train Confusion Matrix:\n"); print(res_tr$cm)
```

```
## Train Confusion Matrix:
```

```

##      Pred
## Actual  0    1
##      0 2267    2
##      1   56 175

```

```
cat("\nValidation Confusion Matrix:\n"); print(res_val$cm)
```

```

##
## Validation Confusion Matrix:

```

```

##      Pred
## Actual  0    1
##      0 1339    5
##      1   53 103

```

```
cat("\nTest Confusion Matrix:\n"); print(res_te$cm)
```

```

##
## Test Confusion Matrix:

```

```

##      Pred
## Actual  0    1
##      0 901    6
##      1   30  63

```